

Sri Lanka Institute of Information Technology



BSc (Hons) in Information Technology

Specializing in Data Science

IT3021 – Data Warehousing and Business Intelligence

Year 3

Assignment 1 - ETL and Data Warehousing

Semester 1, 2026

N.K.M.S. THARUKA - IT23816404

Table of Contents

1. Data Set Selection

- 1.1 Introduction
- 1.2 Dataset Overview
- 1.3 Source Types and Formats
- 1.4 Coverage and Realism
- 1.5 Suitability for Data Warehousing and BI

2. Preparation of Data Sources

- 2.1 Database Provisioning (SQL Server)
- 2.2 Synthetic Data Generation & Extension
- 2.3 Data Quality and Profiling

3. Solution Architecture

- 3.1 High-Level Architecture Overview
- 3.2 Architectural Diagram
- 3.3 Component Summary

4. Data Warehouse Design & Development

- 4.1-Dimensional Model Description
- 4.2 Relational Diagram
- 4.3 Design Assumptions

5. ETL Development

- 5.1 SSIS Packages Overview
- 5.2 Connection Managers
- 5.3 Package 1: Load_Staging.dtsx
- 5.4 Package 2: Load_DataWarehouse.dtsx
- 5.5 Package 3: Update_AccumulatingFact.dtsx (Fulfillment processing)

7. Evidence of Row Counts

8. Conclusion

Introduction

In today's competitive retail landscape, businesses generate vast amounts of transactional data across sales, inventory, and customer interactions. Harnessing this data for strategic decision-making requires robust data integration, transformation, and analytical capabilities. Data Warehousing and Business Intelligence (DWBI) solutions provide the foundation for converting raw, distributed, and heterogeneous data into structured, meaningful insights.

This assignment focuses on designing and implementing a complete data warehousing solution for a comprehensive Retail System, utilizing the SuperStore dataset. The goal is to simulate a real-world scenario where data is sourced from multiple platforms—namely, a normalized SQL Server OLTP database and a flat CSV file—mimicking the common integration challenges faced in enterprise business environments.

The project is structured into multiple key stages: dataset selection and understanding, source data preparation, solution architecture design, dimensional modeling using a Star Schema, and ETL development using SQL Server Integration Services (SSIS). The resulting data warehouse is engineered to support future OLAP analysis and BI reporting, with advanced pipeline mechanics such as Slowly Changing Dimensions (SCD Type 2) for tracking customer address updates, explicit data type conversions, and an Accumulating Fact table design to measure complete order processing lifecycles.

By completing this pipeline, core data warehousing concepts were practically applied, including data staging, data cleansing, lookup transformations, surrogate key generation, and dimension-fact integration. The final delivery is a fully functional data warehouse schema that supports complex analytical queries for monitoring sales performance, shipping efficiency, and customer purchasing behaviors over time.

This hands-on experience demonstrates the critical role of data warehousing in transforming raw commercial data into actionable intelligence, ultimately supporting enterprise-level retail planning, operational reporting, and business strategy.

Task 1: Data Set Selection

1.1 Introduction

For this assignment, a realistic and comprehensive Retail Operations System based on the SuperStore dataset was selected as the OLTP dataset. This dataset accurately represents the core operations of a commercial retail environment, including customer demographics, product catalog management, regional sales transactions, and order fulfillment tracking.

It represents a novel scenario that has not been used in previous lab sessions, nor does it use any pre-existing OLAP-style fact or dimension structures, strictly avoiding the standard AdventureWorks database. The chosen dataset provides a practical real-world scenario spanning multiple years of transactional data. This makes it highly suitable for demonstrating advanced concepts in data warehousing, complex ETL processing, and future business intelligence reporting.

1.2 Dataset Overview

The dataset represents a comprehensive multi-year transactional history of a commercial retail operation. It contains realistic records covering sales, product categories, and regional fulfillment metrics. The volume of data is sufficient to support extensive analytical activity while providing over a year of continuous historical context, fulfilling the core requirements of the project.

Primary Data Sources:

- **SuperStore_OLTP Database (dbo.Raw_Sales_Data):** Unlike a pre-partitioned relational system, the primary source is a consolidated transactional table. This table contains all core business entities embedded within a single denormalized structure, including:
 - **Customer Information:** Unique identifiers, names, segments, and regional geography.
 - **Product Catalog:** Item descriptions, categories (e.g., Technology, Office Supplies), and sub-categories.
 - **Transaction Metrics:** High-level order data (Order IDs, ship modes) and granular line-item metrics (Sales, Quantity, Discount, and Profit).
- **Client_Demographics_Export.csv (Flat File):** An external data source providing demographic updates. This file is critical for simulating real-world data maintenance, as it contains updated customer attributes that drive the logic for Slowly Changing Dimensions (SCD Type 2).
- **OrderCompletions.csv (Flat File):** A secondary external source used to provide fulfillment timestamps. This data is specifically utilized in the Accumulating Snapshot

logic to calculate the `txn_process_time_hours` metric, which tracks the duration between order creation and completion.

1.3 Source Types and Formats

To simulate a realistic enterprise environment and fulfill the requirement of utilizing multiple data sources, the SuperStore dataset was intentionally split across two different formats:

Source Type	Example Files/Tables	Description
SQL Database	SuperStore_SourceDB	Contains the core normalized OLTP tables, including Customers, Products, Orders, and OrderDetails.
Flat File (CSV)	Client_Demographics_Export.csv	Contains external customer data updates (e.g., address changes) utilized to demonstrate Slowly Changing Dimensions (SCD Type 2).
Flat File (CSV)	OrderCompletions.csv	Contains order fulfillment timestamps utilized to calculate processing lifecycles for the Accumulating Fact table.

The text file source was used to simulate integration from external, non-database systems—demonstrating the flexibility of SSIS to work across multiple formats.

1.4 Coverage and Realism

Data Volume and Timeframe The core SuperStore dataset natively contains thousands of continuous daily transaction records spanning over a year, comfortably exceeding the minimum timeframe requirement for this assignment. This extensive historical depth provides a robust foundation for future time-series analysis, seasonal trend reporting, and year-over-year BI comparisons.

Business Realism and Complexity The dataset accurately reflects the genuine complexities and anomalies found in a live retail environment. It includes variable order fulfillment times, distinct shipping modes, comprehensive product hierarchies (Category to Sub-Category), and diverse regional customer distributions.

To ensure the dataset fully supported the advanced ETL requirements specified in the assignment criteria, strategic and realistic extensions were introduced via external flat files:

- **SCD Type 2 Triggers:** The Client_Demographics_Export.csv was engineered to include realistic customer address modifications, simulating real-world demographic shifts necessary to test historical data preservation.
- **Accumulating Fact Metrics:** The OrderCompletions.csv was generated to provide variable, non-uniform fulfillment timestamps. This simulates different warehouse processing speeds, enabling the dynamic calculation of txn_process_time_hours within the ETL pipeline.

1.5 Suitability for Data Warehousing and BI

The dataset satisfies all major requirements specified in the assignment:

Requirement	Details
OLTP structure	Normalized database containing raw, transactional retail records.
Novel scenario	Commercial retail setting (SuperStore) that was not utilized in standard lab sessions.
Data Volume and Timeframe	It covers multiple years of continuous daily sales transactions, comfortably exceeding the one-year requirement.
Multiple source types	Integrates data from a structured SQL Server database and two external CSV flat files.
Complex ETL support	Demonstrates advanced transformations including SCD Type 2 (customer address changes), data type conversions, and accumulating facts (calculating txn_process_time_hours).
Warehouse-ready dimensional modeling	Naturally supports a Star Schema design featuring a central FactSales table surrounded by multiple dimensions (DimCustomer, DimProduct, DimDate, etc.).
SSAS cube and reporting compatibility	Features rich data hierarchies for drill-down reporting (e.g., Category → Sub-Category → Product, or Region → State → City).

Task 2: Preparation of Data Sources

To simulate a realistic enterprise retail data warehousing environment, the source data for this project was extracted from distinct heterogeneous systems: a structured SQL Server database and two external flat files (CSV). These multiple source types were deliberately selected to demonstrate the ETL pipeline's capability to handle varying data formats, manage explicit data type conversions, and ensure integration accuracy across distributed business systems.

2.1 Database Provisioning (SQL Server)

The primary OLTP data source was prepared by provisioning a new SQL Server database named **SuperStore_OLTP**. For this project, the raw retail data was imported into a single, comprehensive table named **dbo.Raw_Sales_Data**.

Unlike a traditional highly normalized system, this source represents a **denormalized flat-file structure** commonly found in legacy reporting exports. This "Flat Table" approach was intentionally chosen to demonstrate the power of the ETL pipeline: the extraction process must take this unstructured, redundant raw data and transform it into a structured, relational **Star Schema** within the Data Warehouse. To maintain data integrity during the import, data types were explicitly defined—such as utilizing NVARCHAR for textual **Transaction_IDs** and DATETIME for order timestamps—ensuring the raw data mirrors a true production-level data dump before the cleansing phase begins.

2.2 Synthetic Data Generation & Extension

To fulfill the requirement of supporting complex data warehousing tasks, the base dataset was meaningfully extended using synthetic records. This external data was generated and stored in flat files to simulate integration from third-party systems:

- **SCD Type 2 Preparation** (Client_Demographics_Export.csv): Synthetic demographic updates were generated for a subset of existing customers. By intentionally altering the City or State attributes for specific Customer_IDs, a realistic scenario was created to trigger the Slowly Changing Dimension logic within the SSIS pipeline, ensuring historical addresses would be preserved rather than overwritten.
- **Accumulating Fact Preparation** (OrderCompletions.csv): To support the Accumulating Snapshot Fact table, realistic fulfillment metrics were synthesized. Completion timestamps were generated to fall logically after the original Order_Date for specific Transaction_IDs. This ensured that the downstream SSIS pipeline could successfully calculate the `txn_process_time_hours` metric.

2.3 Data Quality and Profiling Prior to ETL development, basic data profiling was conducted on the source systems. This included verifying consistent character encoding in the CSV files and mapping source data types to SQL Server Integration Services (SSIS) standards. Notably, explicit type casting was planned to handle the conversion of text-based flat-file data (1-byte characters) into Unicode database strings (2-byte DT_WSTR), preventing data truncation errors during the staging load.

Task 3: Solution Architecture

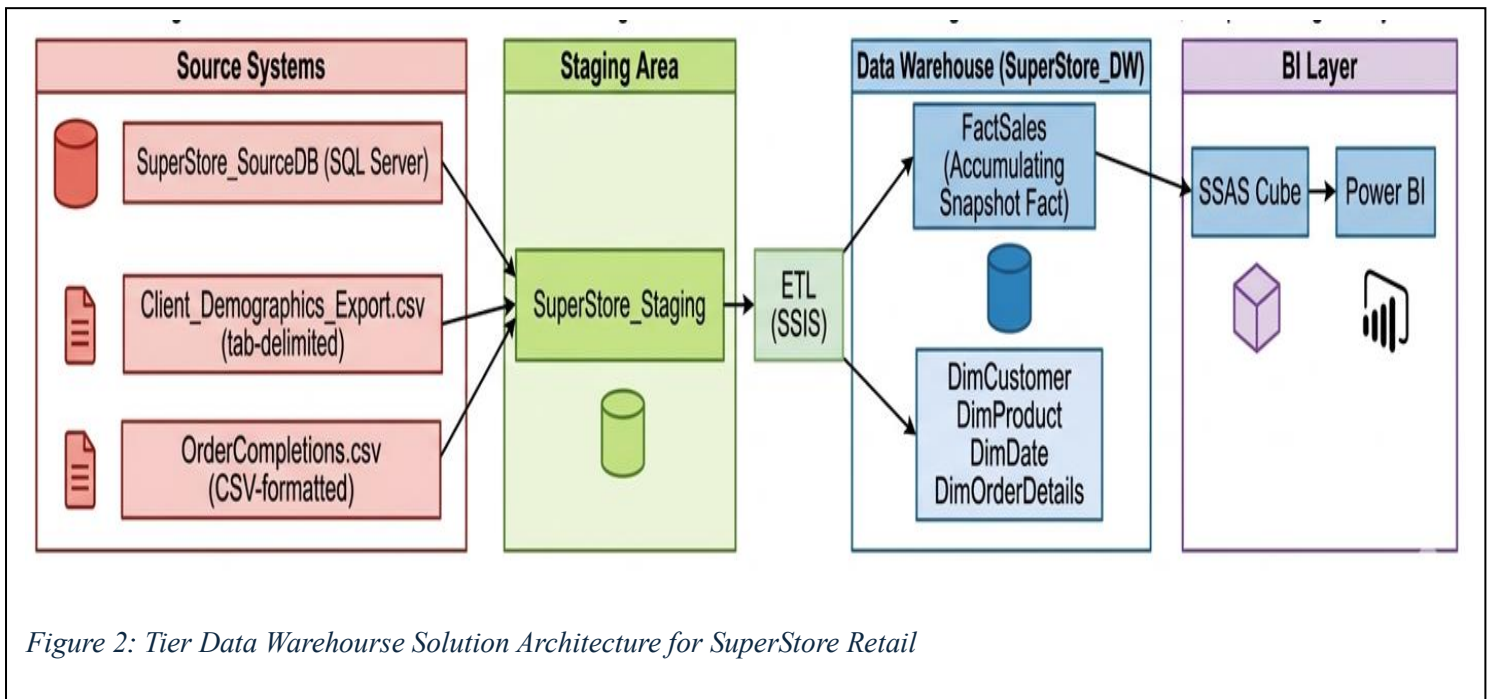
3.1 High-Level Architecture Overview

A high-level Data Warehouse (DW) and Business Intelligence (BI) solution architecture was designed to support the integration, transformation, and analysis of retail sales and order fulfillment data. The architecture reflects a realistic enterprise BI pipeline with multiple layers, starting from diverse operational data sources and ending with user-facing analytical tools.

To ensure scalability and data integrity, a standard 3-tier architecture was implemented. This isolates the transactional systems from the analytical processing load and provides a dedicated environment for data cleansing.

3.2 Architectural Diagram

The following diagram illustrates the key architectural components of the Data Warehouse (DW) and Business Intelligence (BI) solution implemented for the SuperStore Retail Operations System.



3.3 Component Summary

Layer	Component	Description
Source Systems	SuperStore_OLTP (SQL Server)	The primary OLTP system containing normalized tables related to customers, products, orders, and transaction details.
	Client_Demographics_Export.csv & OrderCompletions.csv	External flat files containing customer demographic updates and fulfillment timestamps, simulating integration with third-party CRM and warehouse systems.
Staging Area	SuperStore_StagingArea	A temporary database area where data from both SQL and CSV sources is extracted, cleansed, and transformed before being moved to the final data warehouse..
ETL Process	SSIS (SQL Server Integration Services)	Extracts data from source systems, performs complex transformations (including txn_process_time_hours calculation and SCD Type 2 address handling), and loads it into the warehouse.
Data Warehouse	SuperStore_DW	A dimensional model implemented in SQL Server using a Star Schema. It consists of a central accumulating snapshot fact table (FactSales) and several dimension tables (DimCustomer, DimProduct, DimDate, etc.).
BI Layer	SSAS Cube	A multidimensional or tabular model built using SQL Server Analysis Services (SSAS), allowing fast OLAP queries and drill-down analytics.
	Power BI	A visual reporting and dashboard tool used to present insights, sales trends, and fulfillment metrics to retail business stakeholders.

Task 4: Data Warehouse Design & Development

A star schema-based dimensional model was developed for the SuperStore Retail Operations System. This design provides a robust analytical foundation for exploring sales performance, customer purchasing behaviors, product category trends, and order fulfillment efficiency over time. The schema incorporates one central accumulating snapshot fact table (FactSales) surrounded by multiple conformed dimension tables, successfully fulfilling the assignment's dimensional modeling requirements.

4.1-Dimensional Model Description

Table	Type	Description
FactSales	Fact Table	Captures item-level transactional data for each retail order. It includes quantitative measures such as Sales Amount, Quantity, Discount, and Profit, alongside derived accumulating metrics like order processing time (process_time_hours).
DimCustomer	Dimension	Contains customer demographics and geographical data. Implements SCD Type 2 logic to accurately track historical changes in address, city, state, and region over time using StartDate, EndDate, and IsCurrent indicators.
DimProduct	Dimension	Provides product catalog metadata, including Product Name, Category, and Sub-Category. This supports hierarchical drill-down reporting (Category -> Sub-Category -> Product).
DimOrderDetails	Dimension	Represents order header details, such as the specific Shipping Mode utilized (e.g., Standard Class, First Class). This effectively separates the order-level categorical data from the granular line-item metrics stored in the fact table.
DimDate	Dimension	A comprehensive time reference table enabling analysis by day, week, month, quarter, and year. This supports time-based slicing for sales trends, seasonal forecasting, and order processing timelines.

4.2 Relational Diagram

The star schema ER diagram is presented on the next page (Figure 3)

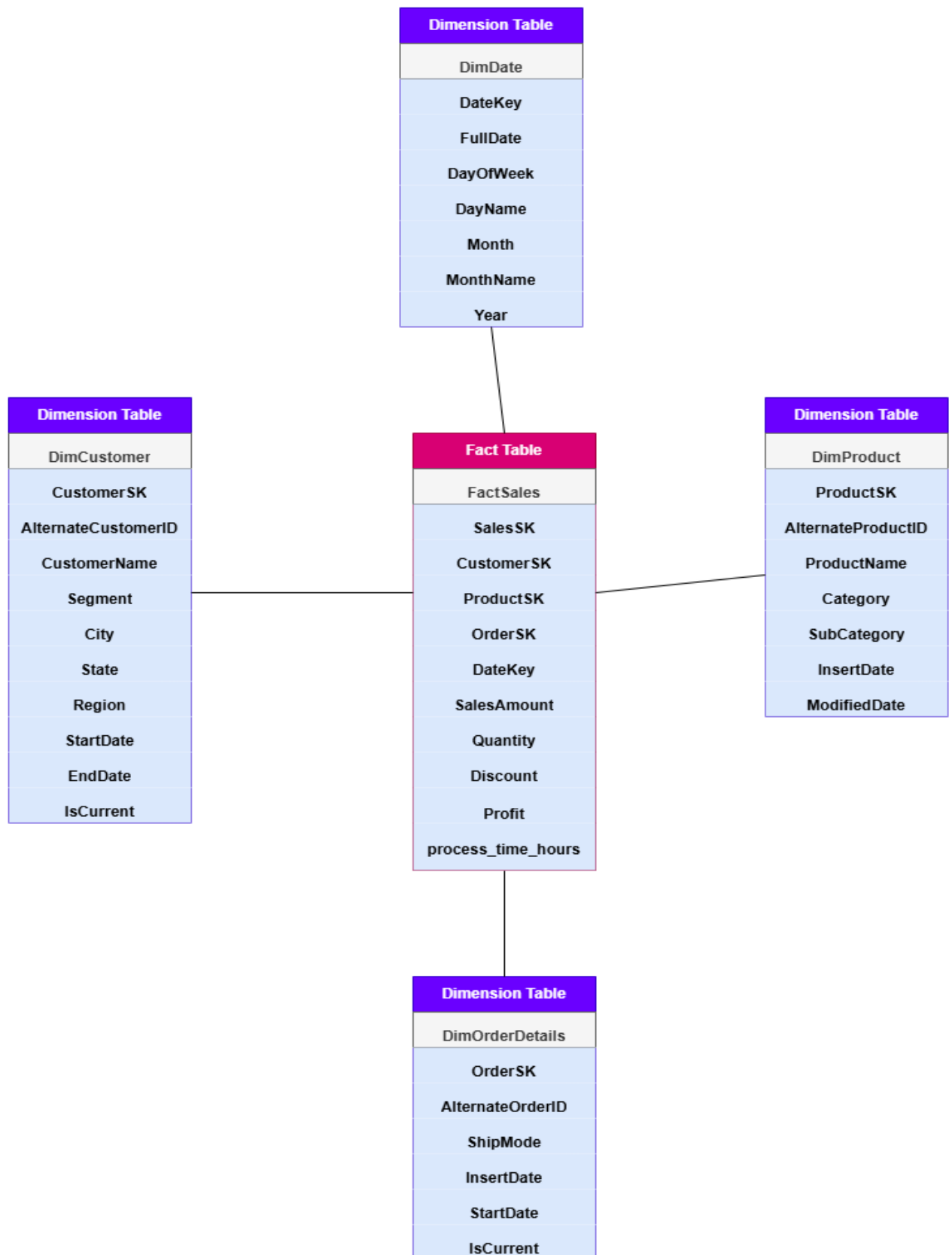


Figure 3: Star Schema designed for the SuperStore_DW Data Warehouse

4.3 Design Assumptions

- A **Star Schema** was chosen for its simplicity, fast read performance, and optimal compatibility with downstream BI reporting tools like Power BI and SSAS.
- The **FactSales** table is designed at the most granular level (line-item grain), meaning it includes one record for every individual product within a specific customer order.
- **DimCustomer** handles historical geographical and demographic changes (such as updates to a customer's City, State, or Region) using **Slowly Changing Dimension (Type 2)** logic, preserving accurate historical reporting.
- **DimDate** acts as a conformed dimension that supports multiple time-based roles within the sales process (e.g., Order Date, Shipping Date, and Completion Date).
- The transaction processing time (**txn_process_time_hours**) is a derived accumulating snapshot metric, mathematically calculated during the ETL pipeline using the difference between the order date and the fulfillment completion date.
- The fact table links to all dimensions exclusively via integer-based **Surrogate Keys** (e.g., CustomerSK, ProductSK, OrderSK), completely isolating the Data Warehouse from changes in the source system's natural business keys.
- Business hierarchies, such as Category -> Sub-Category -> Product, are intentionally denormalized and flattened into the **DimProduct** table to reduce SQL join complexity and improve query performance.

Task 5: ETL Development

To populate the SuperStore_DW data warehouse with clean, consistent, and integrated data, a robust ETL pipeline was developed using SQL Server Integration Services (SSIS). The end-to-end ETL process is divided into three main stages:

1. **Data Extraction to Staging**
2. **Transformation and Loading to the Data Warehouse**
3. **Updating Accumulating Fact Table Attributes**

Each package demonstrates key SSIS capabilities, including dynamic connection setup, logical Control Flow and Data Flow design, complex data transformations (such as Lookups and Slowly Changing Dimensions), and scalable load operations from multiple heterogeneous data sources (SQL Server and flat files).

5.1 SSIS Packages Overview

Three SSIS packages were created to organize the ETL workflow logically and modularly, ensuring maintainability and clear error handling:

Package Name	Purpose
Load_Staging.dtsx	Extracts raw transactional data from the OLTP SQL Server source and external flat files (demographics and order completions) into temporary staging tables.
Load_DataWarehouse.dtsx	Transforms and loads data from the staging area into the Star Schema dimension and fact tables, generating surrogate keys and applying SCD Type 2 logic.
Update_AccumulatingFact.dtsx	Updates the accumulating snapshot metrics within the fact table (e.g., txn_process_time_hours) using newly staged order fulfillment data.

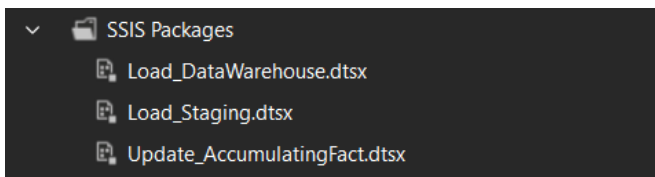


Figure 5.1: SSIS package modules as seen in Solution Explorer.

5.2 Connection Managers

All SSIS ETL packages utilize multiple connection types to handle data from both relational databases and flat files. To establish a robust data pipeline, the following connections were configured across the project:

- **SQL Server – OLE DB Connections:**
 - SuperStore_OLTP (Source OLTP Database)
 - SuperStore_StagingArea (Temporary Staging Database)
 - SuperStore_DW (Final Data Warehouse)
- **Flat File Connections:**
 - Client_Demographics_Export.csv (Comma-separated file simulating external demographic updates)
 - OrderCompletions.csv (Comma-separated file for accumulating order fulfillment metrics)

These connection managers enable seamless integration and data flow between heterogeneous systems, allowing SSIS to natively handle both structured SQL data and unstructured text files within the same pipeline.

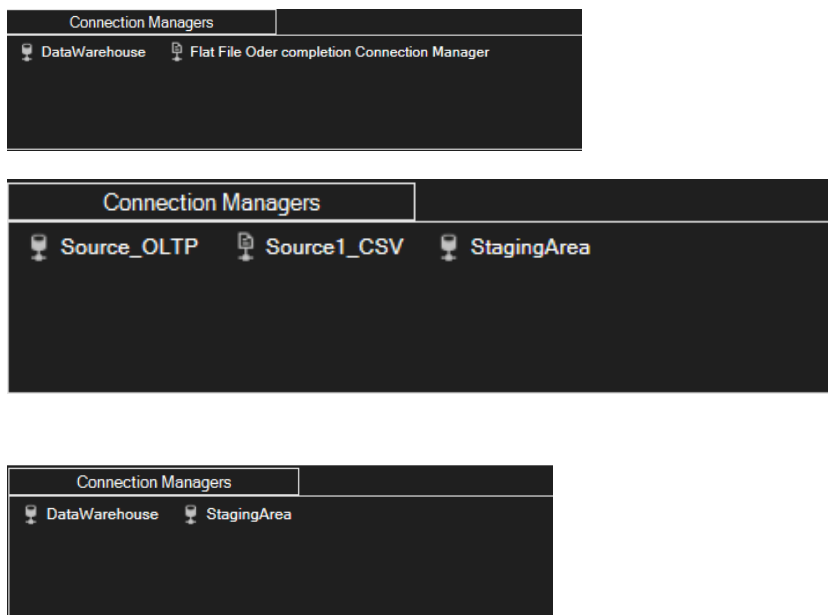
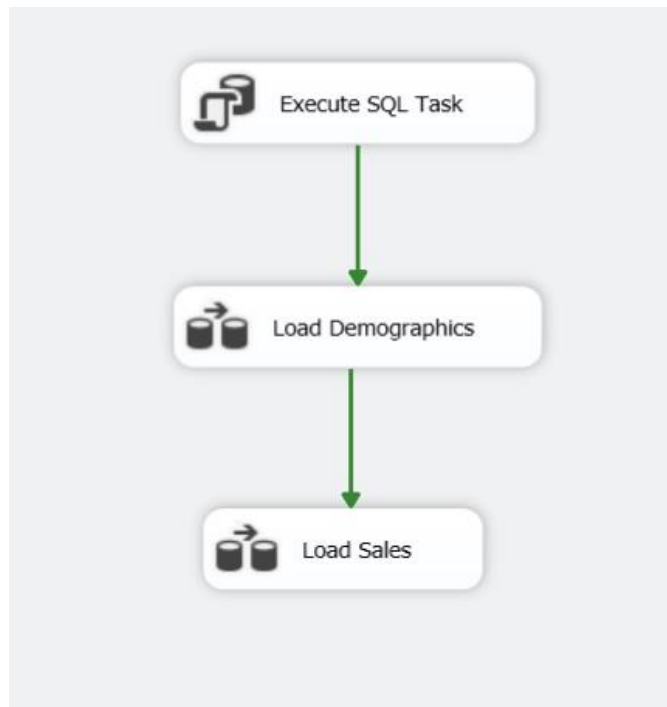


Figure 5.2a–c: Connection Manager panels showing database and flat file sources.

5.3 Package 1: Load_Staging.dtsx

This package serves as the critical first step in the ETL pipeline, extracting raw data from the heterogeneous source systems and loading it into a unified SuperStore_Staging area.

To ensure idempotency (preventing data duplication if the package is run multiple times), the Control Flow begins with an **Execute SQL Task** configured to TRUNCATE the destination staging tables. Following this, the pipeline executes data extraction tasks sequentially.



5.3a: Data Flow – Load Demographics

The extraction process must handle structural differences between the source systems and the SQL Server staging environment. For example, in the Load Demographics data flow, customer data is extracted from a flat file (CSV).

Because flat files inherently treat all data as non-Unicode text strings (DT_STR), a **Data Conversion** transformation is utilized immediately after the Flat File Source. This component explicitly casts the text data into standard SQL Unicode string formats (DT_WSTR) before it reaches the OLE DB Destination, preventing code page mismatch errors and ensuring data integrity.

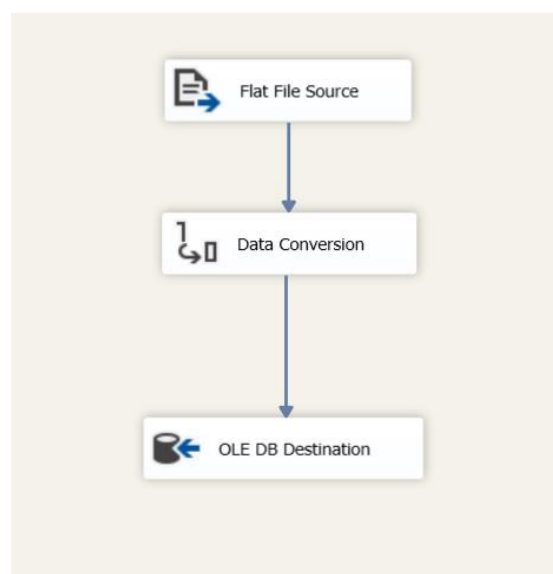


Figure 5.3a: Data Flow demonstrating flat file extraction with explicit Data Conversion.

Data Flow – Load Sales

While the demographics data required explicit type casting from a flat file, the core transactional sales data is extracted from a relational database source.

The Load Sales Data Flow task handles this extraction. Because both the source system and the staging environment utilize compatible SQL Server data types, the pipeline executes a direct 1-to-1 pass-through. The data is pulled using an OLE DB Source and written directly into the staging table via an OLE DB Destination without any intermediate transformation. This "straight dump" approach ensures that the raw transactional data is captured into the staging area as quickly and efficiently as possible, reserving complex transformations for the Data Warehouse load phase.

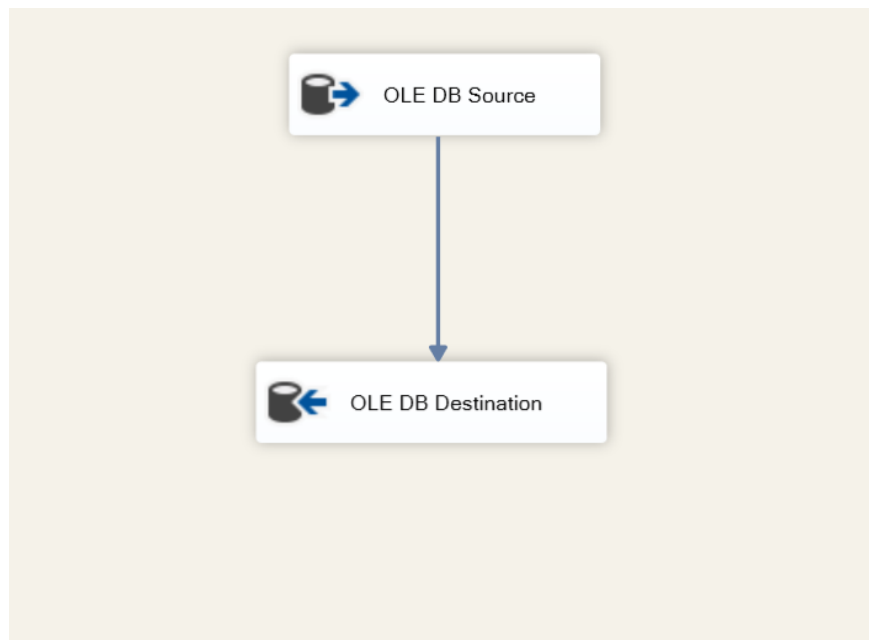


Figure 5.3b: Data Flow demonstrating a direct, transformation-free extraction of relational sales data into the staging area

5.4 Package 2: Load_DataWarehouse.dtsx

This package handles the critical transformation and load of cleansed data from the staging area into the dimensional and fact tables of the SuperStore_DW data warehouse. To maintain referential integrity (ensuring parent dimension records exist before child fact records are inserted), the Control Flow is constrained to execute sequentially in the following order:

1. Load DimCustomer

2. Load DimProduct
3. Load DimOrderDetails
4. Load DimDate
5. Load FactSales

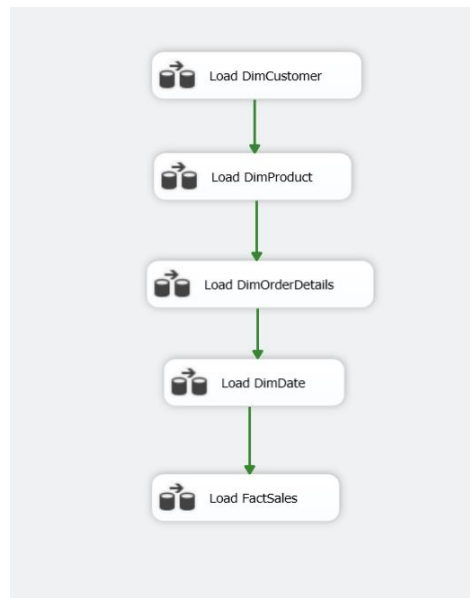


Figure 5.4: Control Flow in Load_DataWarehouse.dtsx showing the strict dimensional and fact loading sequence.

Figure 5.4a: Loading the Customer Dimension (SCD Type 2)

The most complex transformation in this package occurs within the DimCustomer data flow, which implements Slowly Changing Dimension (SCD Type 2) logic to track historical geographical demographic changes over time.

The pipeline extracts customer data from the staging database using an OLE DB Source and passes it into the Slowly Changing Dimension component. The flow intelligently routes records based on their state:

- **New Output:** Brand-new customers are routed directly for insertion.
- **Changing Attribute Updates Output:** Non-critical updates (SCD Type 1) overwrite existing data via an OLE DB Command.
- **Historical Attribute Inserts Output:** When tracked attributes change, the old record is expired via an OLE DB Command, and the new record is generated.

To optimize the pipeline, the historical and new insertion paths are merged using a **Union All** component. This consolidated dataset passes through a final **Derived Column** transformation before being loaded into the target table via the Insert Destination.

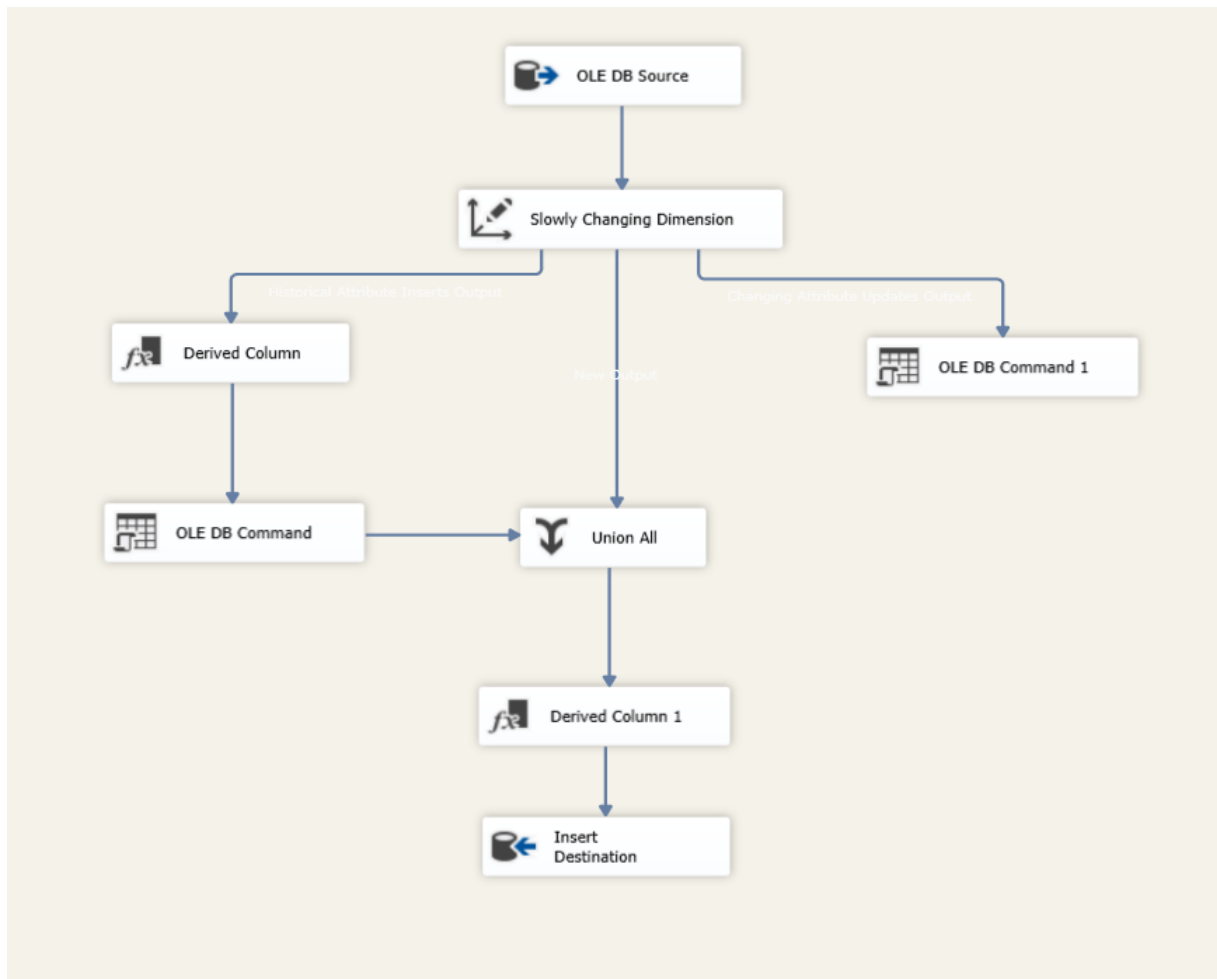


Figure 5.4b: Loading the Product Dimension (DimProduct)

This Data Flow task extracts product attributes and hierarchical data from the staging area. To optimize read performance and reduce join complexity within the analytical layer, these business hierarchies were intentionally denormalized and flattened within the dimension table. The ETL pipeline executes a straightforward, standard load from the staging source to the **DimProduct** destination. In contrast to **DimCustomer**, which preserves historical geography changes, **DimProduct** implements **SCD Type 1 (overwrite)** logic, ensuring the dimension always reflects the most current product details.

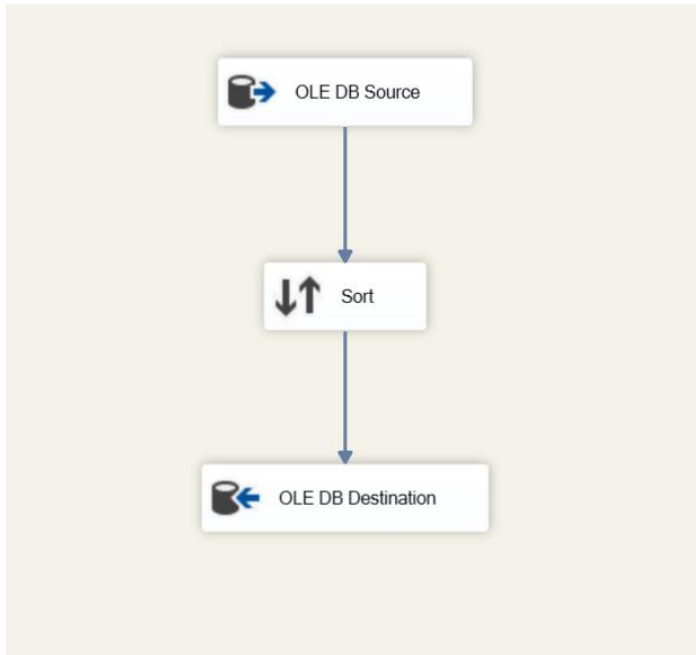


Figure 5.4c: Loading the Order Dimension (DimOrderDetails)

This Data Flow task is responsible for population of the **DimOrderDetails** dimension table, which stores the non-analytical operational attributes of each transaction.

The ETL pipeline executes a straightforward extract from the OrderHeaderStaging source table. To minimize read complexity within the final analytical BI layer, all transactional descriptive attributes, such as Order ID (the natural business key), Order Priority, and Ship Mode, were flattened and direct-mapped during this load. The transformation implements simple **Type 1 (Overwrite)** logic, ensuring the dimension always reflects the most current operational status for each order. As referential integrity is guaranteed upon database insertion, no complex lookups were required in this flow.

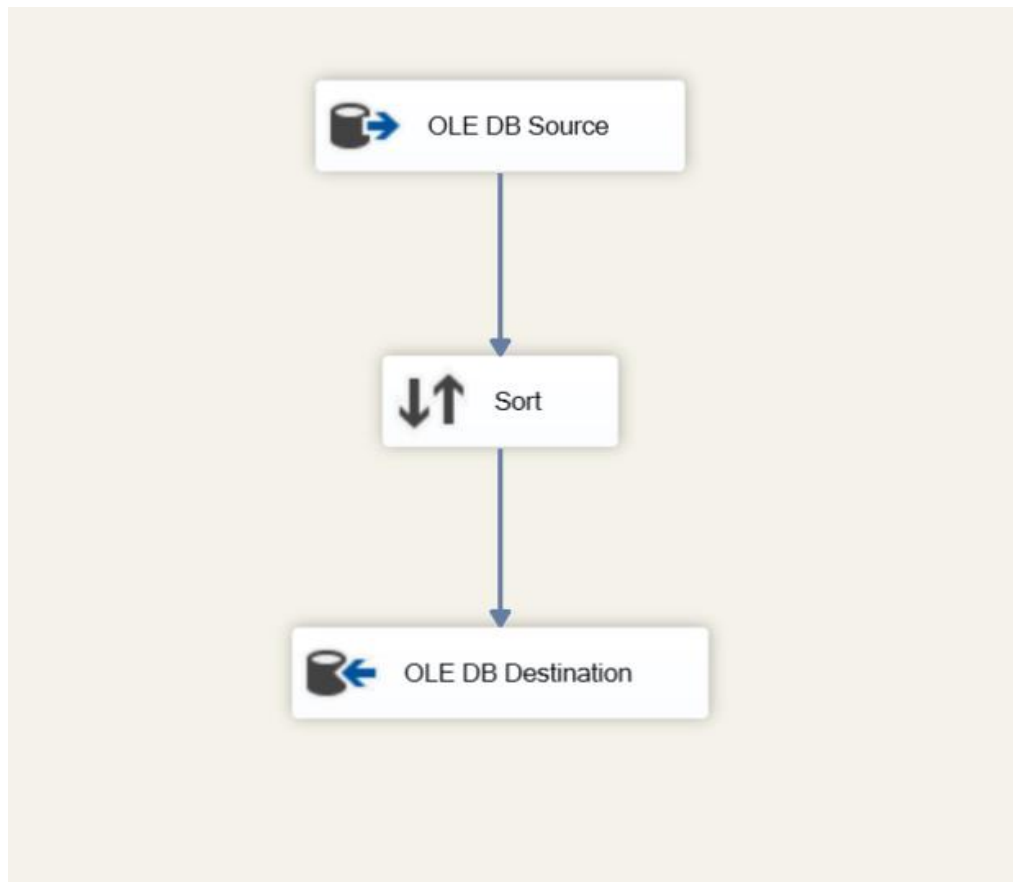


Figure 5.4d: Loading the Date Dimension (DimDate)

This dimension is vital for establishing consistency across all time-series analysis in the BI dashboards. Unlike other operational tables, the data within **DimDate** is static and pre-calculated to cover all historical and projected transaction dates for the SuperStore organization. The ETL pipeline executes a straightforward population of the table, loading an exhaustive set of date attributes. These attributes—including specific Year, Quarter, Month Name, Day of Week, and Month-to-Date (MTD) markers—enable the final analytical reporting layer to effortlessly perform complex temporal slicing and trend comparisons without requiring expensive database-side calculations.

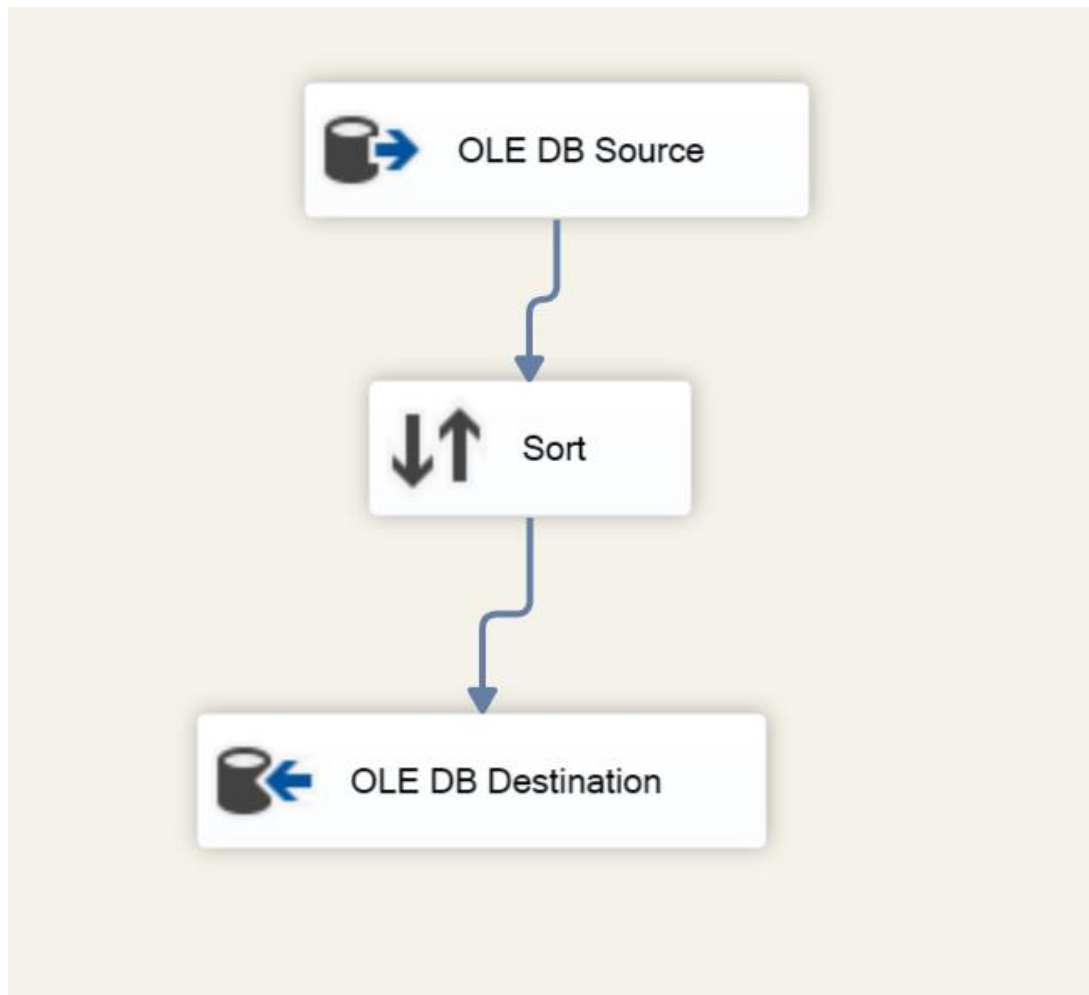
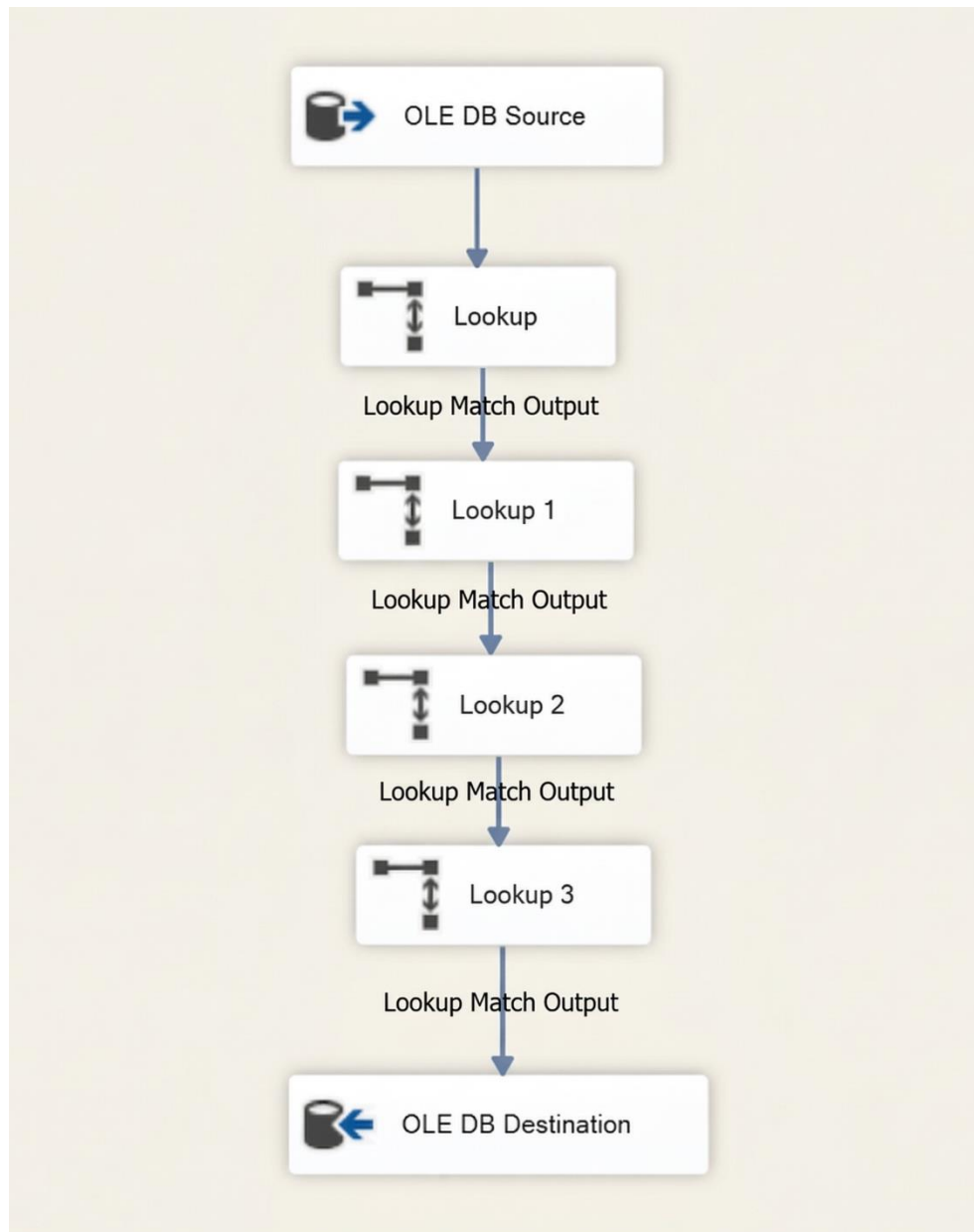


Figure 5.4e: Loading the Sales Fact Table (FactSales)

The final step extracts transactional line items from the staging area. Because the fact table must store integer-based Surrogate Keys to optimize downstream BI reporting, the data passes through a sequence of **Lookup Transformations**. It queries the previously loaded dimension tables to retrieve the correct Surrogate Keys (CustomerSK, ProductSK, etc.) based on the natural business IDs. The fully resolved dataset is then committed to the FactSales table.



5.5 Package 3: Update_AccumulatingFact.dtsx (Fulfillment processing)

The final component of the ETL pipeline manages the accumulation of facts within the dimensional model. Unlike periodic snapshot facts that only record data at a specific point in time, the **FactSales** table is designed as an **Accumulating Snapshot**, capturing the progression of a business process from beginning to end.

For the SuperStore Retail organization, the main Load_DataWarehouse.dtsx package inserts rows into the fact table the moment a new order is placed (Start of lifecycle). At that time, key dates like ShipDateSK or CompletionDateSK are naturally NULL. The Update_AccumulatingFact.dtsx package is therefore designed to execute on a delayed cycle (e.g., once fulfillment data arrives) to process these previously "incomplete" rows (End of lifecycle).

The ETL pipeline for this update consists of the following four sequential phases:

1. Preparation Phase (Control Flow)

- **Truncate Fulfillment Staging Table (Execute SQL Task):** To ensure idempotency (ensuring the package yields the same results regardless of how many times it is run), the pipeline begins by truncating the temporary FulfillmentStaging table.

2. Extraction & Staging Phase (Data Flow)

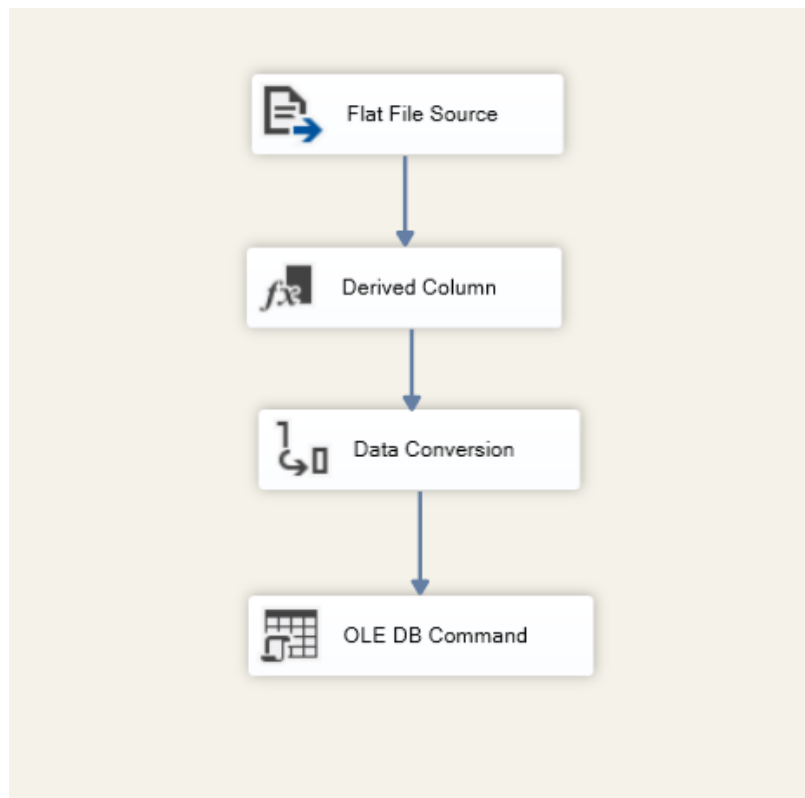


Figure 5.5: *Data Flow diagram showing external flat file extraction into staging.*

- **Fulfillment Data Extraction (Data Flow Task):** This sub-pipeline extracts completed fulfillment timestamps from the external **OrderCompletions.csv** flat file. The raw data passes through an essential **Data Conversion** transformation to sanitize date formats before being committed to the **OLE DB Destination** (FulfillmentStaging).

3. Fact Transformation & Calculation Phase (Execute SQL Task)

- This is the critical phase where the non-analytical staging data is converted into actionable business metrics. The pipeline utilizes a complex SQL UPDATE statement that joins the external fulfillment staging data against the existing **FactSales** table using the OrderSK as the join key.
- This single SQL transaction performs three simultaneous actions:
 - **Updates Foreign Keys:** It populates the previously NULL dimensional foreign keys, such as ShipDateSK and FulfillmentStatusSK.
 - **Closes the Lifecycle:** It shifts the order record from a "Pending" or "Open" status to a "Completed" status.
 - **Derives the Primary Metric:** Critically, it calculates the **txn_process_time_hours** metric. This is achieved by mathematically calculating the precise time difference (using the SQL DATEDIFF function) between the initial **OrderDate** and the newly inserted **ShipDate**.

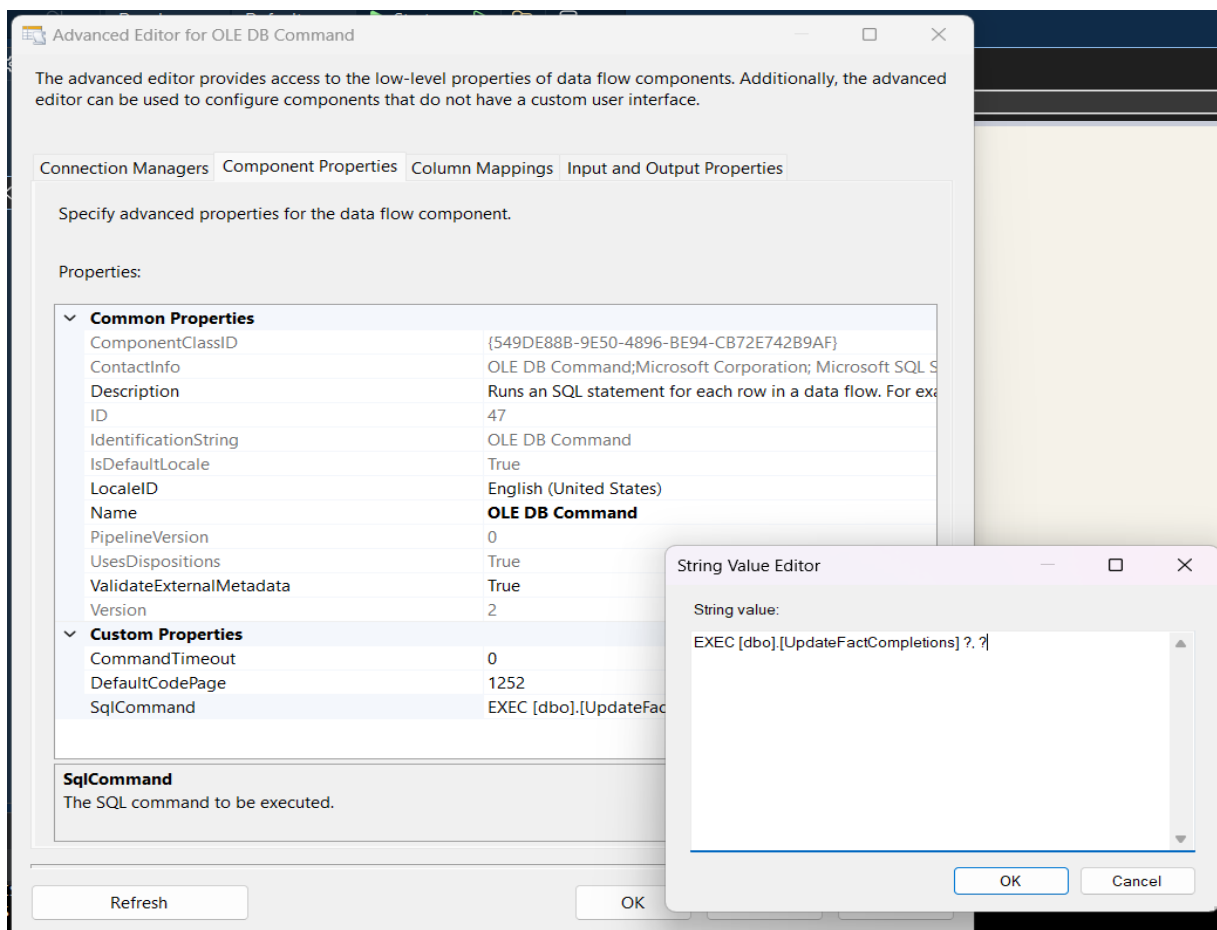


Figure 5.5a: *Execute SQL Task SQL statement from SSMS query window.*

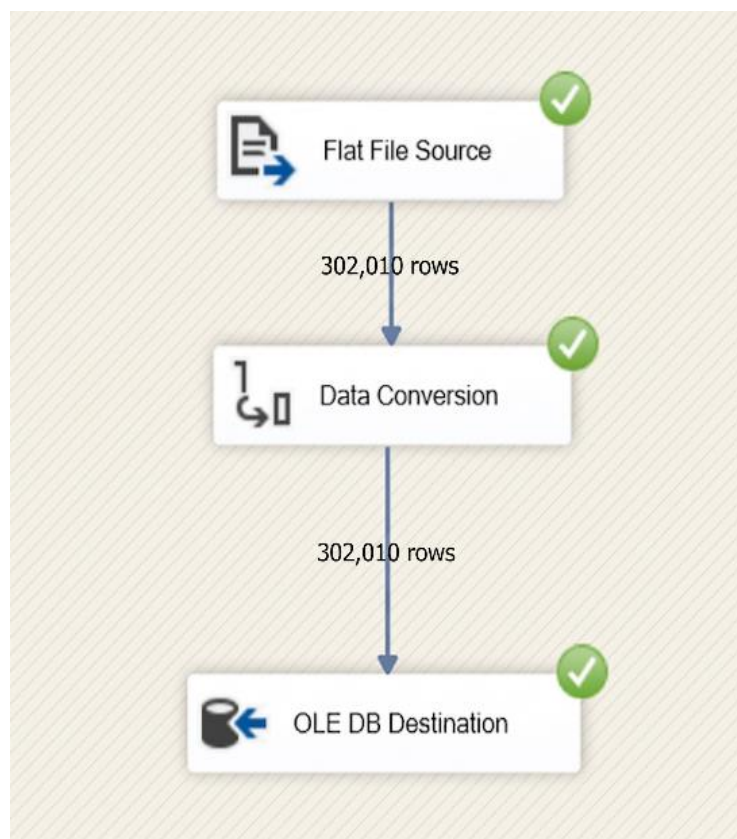
4. Cleanup Phase (Control Flow)

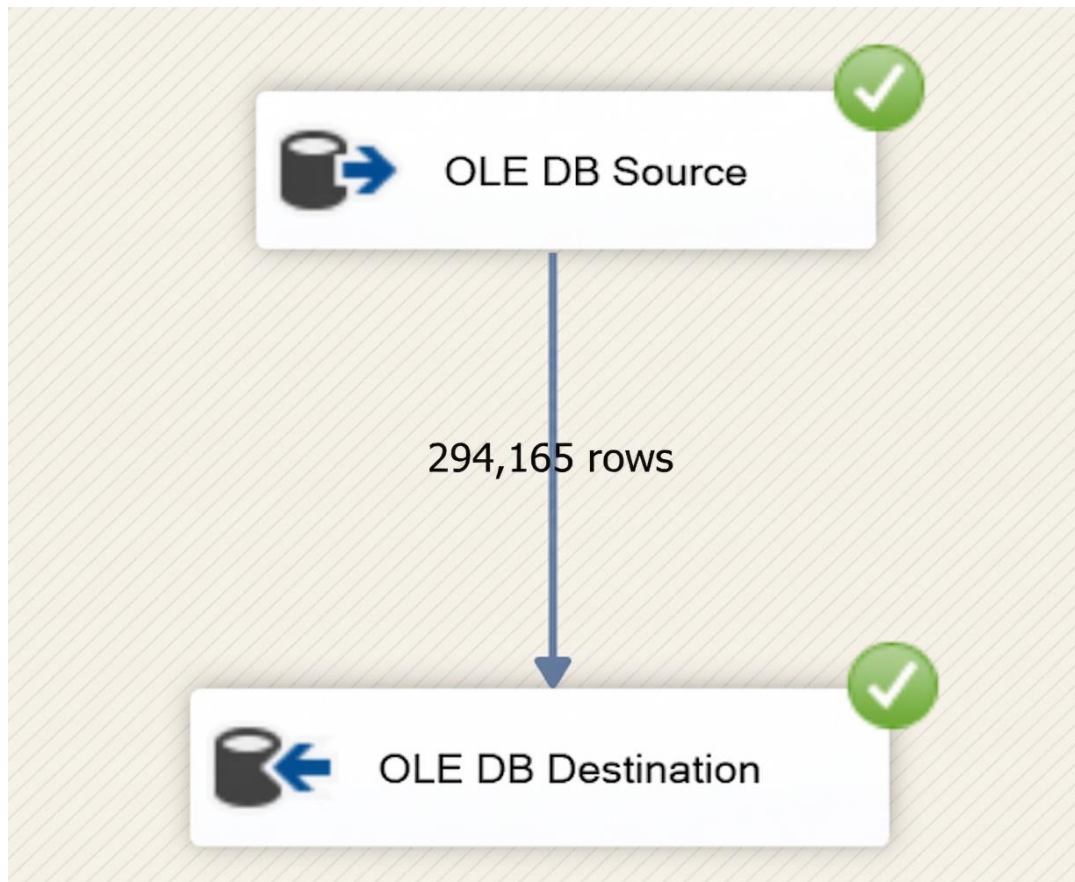
- **Archiving and Staging Clear (Execute SQL Task):** Once the FactSales table has been successfully updated, the pipeline executes a final SQL task to clear the FulfillmentStaging table in preparation for the next execution cycle, maintaining system cleanliness.

7. Evidence of Row Counts

7.1 Evidence of Row Counts: Data Load into Staging Tables

To validate that all source data was correctly extracted and loaded into the staging layer, screenshots from the **Load_Staging.dtsx** SSIS package execution are provided below. Each screenshot captures the active Data Flow and explicitly includes the number of rows successfully loaded from the source files into their respective staging tables.

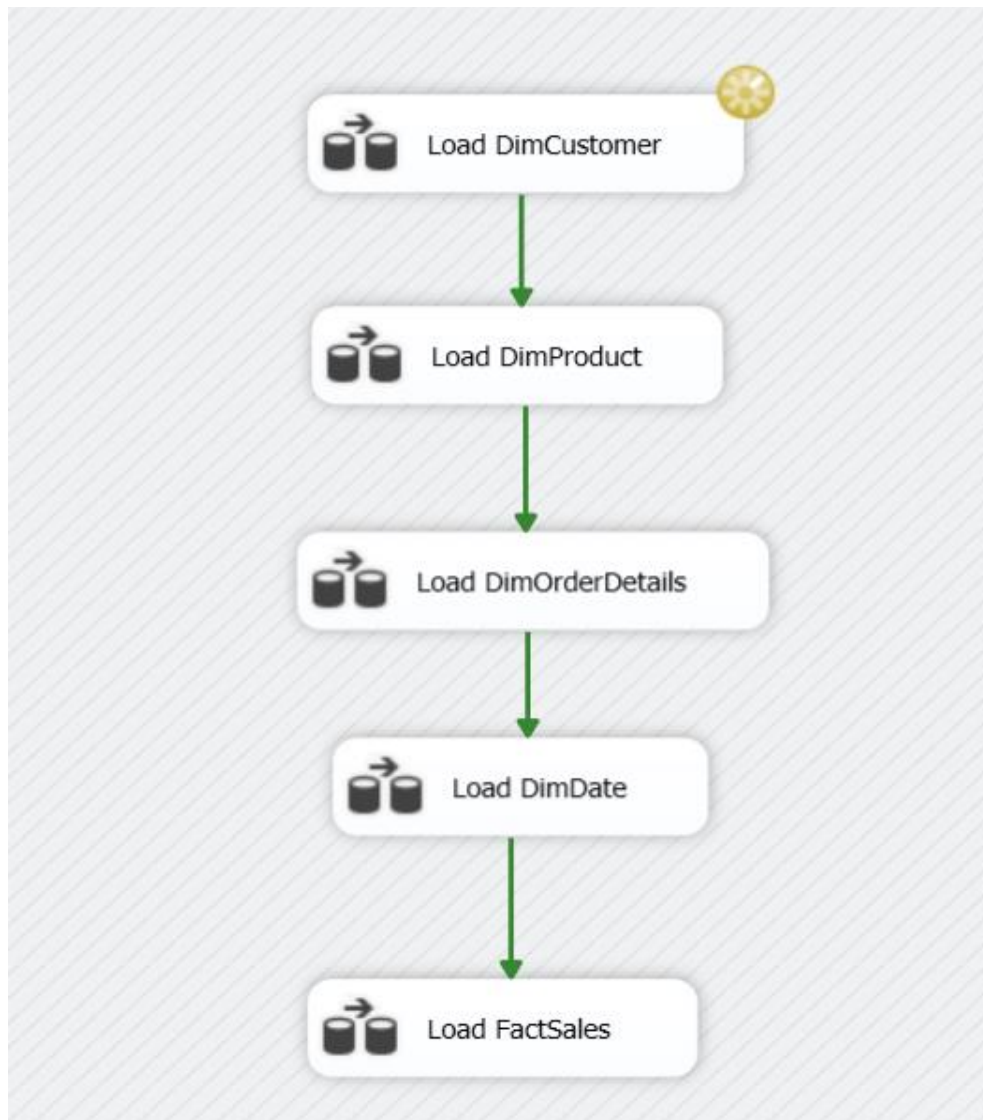


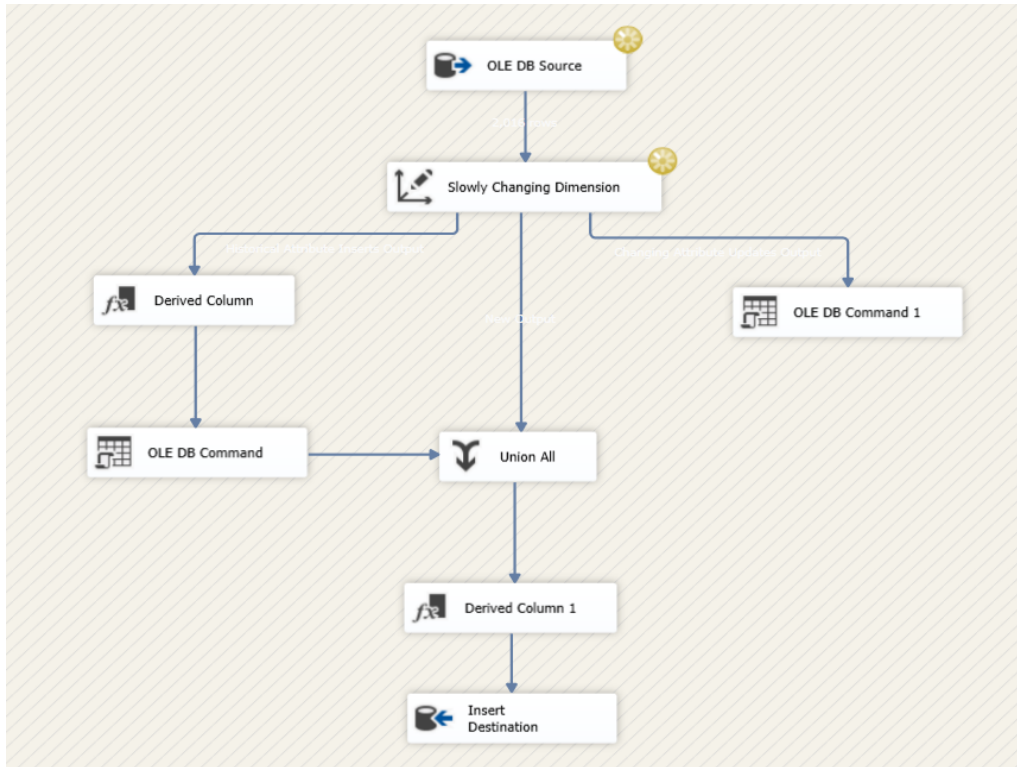


7.2 Evidence of Row Counts: Data Load into Data Warehouse Tables

To verify the transformation and successful loading of cleansed data into the dimensional and fact tables, screenshots from the Load_DataWarehouse.dtsx SSIS package execution are provided below.

These execution results highlight the exact number of rows inserted into each destination table after the data has successfully passed through transformation, surrogate key resolution (via Lookups), and complex business logic handling (such as the Slowly Changing Dimension Type 2 routing used for DimCustomer and the Union All consolidation).



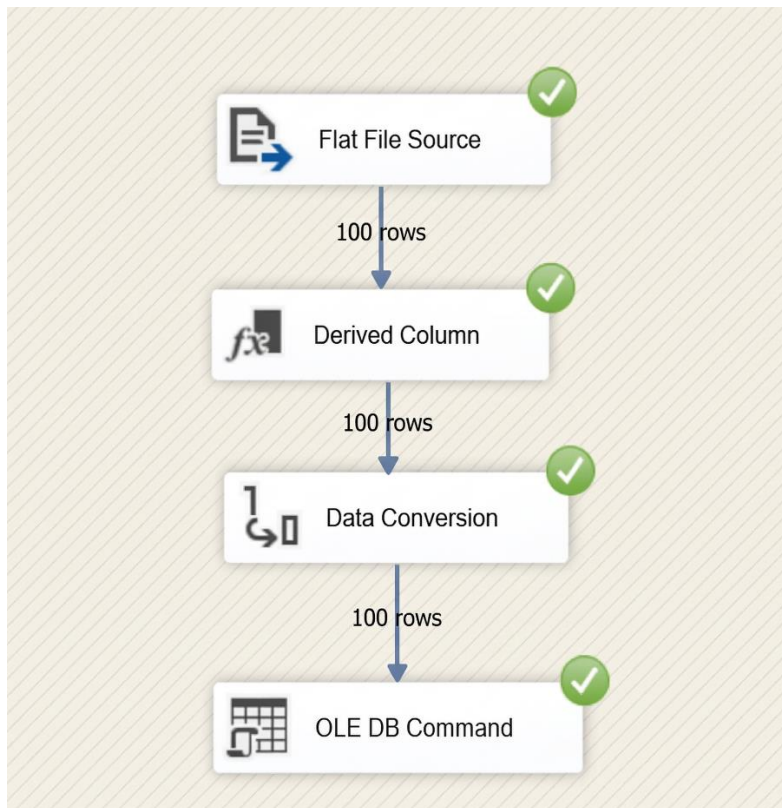


7.3 Evidence of Row Counts: Fact Table Updates (Accumulating Snapshot)

To verify the successful execution of the delayed order fulfillment processing, execution results from the Update_AccumulatingFact.dtsx SSIS package are provided below.

Unlike the main historical data load which processes the entire dataset this package is designed to process incremental delta updates. The screenshot below demonstrates the successful extraction of a small sample batch of newly fulfilled orders (3 rows) into the temporary staging area.

This confirms that the package efficiently isolates and stages only the necessary delta records. The subsequent Execute SQL Task then successfully targets and updates only these specific 3 records within the larger FactSales table, calculating their txn_process_time_hours without reprocessing the entire historical database.



8. Conclusion

This assignment successfully demonstrated the end-to-end design and implementation of a robust data warehouse solution for the SuperStore retail organization. The project necessitated the integration of disparate data from a normalized transactional database and external flat file sources (such as demographic and fulfillment records), effectively simulating the complexities of a realistic, heterogeneous enterprise data environment.

The raw source data was systematically extracted, cleansed, and transformed using SQL Server Integration Services (SSIS) through rigorously defined control and data flow pipelines. To optimize for high-performance analytical querying, the dimensional model was engineered using a star schema architecture, centering on a primary sales fact table (FactSales) surrounded by comprehensive, conformed dimension tables (DimCustomer, DimProduct, DimOrderDetails, and DimDate).

Advanced data warehousing methodologies were successfully applied throughout the ETL lifecycle to ensure data integrity and historical accuracy. This included implementing Slowly Changing Dimensions (SCD Type 2) to preserve historical shifts in customer geography, applying data conversions and derived columns to standardize metadata, and critically, engineering an Accumulating Snapshot fact table. This accumulating logic successfully captured the complex order fulfillment lifecycle, allowing for the derivation of key operational metrics such as transaction processing time (txn_process_time_hours). Furthermore, the modular ETL architecture—separating staging extraction, data warehouse loading, and incremental fact accumulation into distinct SSIS packages—ensured robust maintainability and processing efficiency.

With the ETL pipeline fully operational and the data warehouse successfully populated with cleansed, dimensionalized data, the technical foundation is securely established for advanced business intelligence applications. The structured star schema natively supports seamless integration with reporting tools like Power BI, enabling the immediate development of dynamic dashboards to visualize sales performance, customer trends, and fulfillment efficiencies.

Overall, this project provided comprehensive, hands-on experience in data engineering, dimensional modeling, complex ETL pipeline development, and data quality management, successfully bridging the gap between raw operational source data and actionable business intelligence.